LOGICAL ARCHITECTURE · BASELINE V2 · DRAFT

Logical architecture, target design

* marks target-design items that the running demo implements in a thinner form (see notes)

Plant edge and intake

Tool fleet, three plants *
Etch, deposition, strip, clean tools
Sensors via plant IoT gateways

Offline instruments
Oscilloscope, RF meter printouts
Upload telemetry, read and confirmed

Operator report
Manual ticket with tool and sensor

Quality and supplier events
QMS nonconformance, supplier notice

Simulator control plane *
Scenarios, fault injection, hard and soft reset, replay vs live

Event backbone (single entry point)

Telemetry stream *
Azure Event Hubs (IoT Hub ingress)
Partitioned by tool, ordered

Command and event topics *
Azure Service Bus topics
System and application commands

Message envelope
eventId, type, correlationId, causationId, parentEventId, sequence, source, site, tool, time, payload, schema version

Command catalog
Defined as data: who may issue it and which events it emits

Event log
Append-only, indexed by correlation
Rebuilds every causal chain

Application layer, deterministic

SPC monitoring
Control limits, Western Electric rules

Business rules engine
Ticket creation, escalation, routing
Rules and process maps are data

Lifecycle state machines
Alert, ticket, investigation, approval, CAPA

Evidence gate and grounding check
Weak or conflicting evidence escalates to a person

Approval and action service
Human approves before any CAPA

Agent layer, probabilistic

Investigation orchestrator
Microsoft Agent Framework

Investigation agent
Read-only tools over enterprise systems and documents below

Resolution agent
Ranks root cause, no tools

Model gateway
Routing, retries, daily spend cap
Azure OpenAI gpt-5.4-mini, replay

Enterprise context

Mocked enterprise systems
MES, QMS, PLM, ERP, maintenance

Document store *
SOPs, FMEA, bulletins, search index

Data stores

Metadata store *
Company, site, capability, line, tool, parameter, limits, rules, process maps
Relational (Azure SQL)

Telemetry store *
Time series (Azure Data Explorer)
Retention policy, fast windows

Transaction store *
Alerts, tickets, investigations, approvals, CAPA, audit (Azure SQL)

Causal graph *
Event lineage and genealogy
Graph store, optional (Cosmos DB)

Experience

Meridian FabOps console
Sites, fleet, alerts, tickets, investigation, approval, CAPA, rules and process, causal trace

Live push
SignalR fed by bus subscriptions

Publish and subscribe on the backbone

Every layer is a bus participant
Rules, SPC, lifecycle, agents, stores and live push subscribe to typed topics and publish results with causationId set to the triggering event, so one fault command fans out into readings, alert, ticket, investigation steps, approval and CAPA, all on one correlationId.

NOTES

* The running demo is intentionally thin. One Azure Container App, one replica, hosts the API, simulator and agents.
* Demo bus: in-process, same envelope and command catalog (ADR-008). Measured: 7-day nominal seed, 466,662 readings in 7 s (~66k events/s).
* Demo stores: metadata in YAML loaded at startup (ADR-009); telemetry, event log and transactions in SQLite; 7-day chart read under 1 ms.
* Demo tools: three simulated plants; only the Manufacturing capability is simulated. Document search runs in process.
Every telemetry reading, alert, ticket, investigation step, approval, CAPA and command travels through the event backbone.

Meridian Precision Systems · FabOps

Logical architecture, target design

Prepared by StatusNeo with Common Consulting · Draft 2026-10-15

Sheet ARCH D 36 x 24 in · Fictional company, synthetic data

Copyright (c) 2026 Bryan C. Weis and Common Consulting Company